AyaTrace

Final Execution Plan & Research Runbook

Frozen scope • proofread v1.1 • 5 November 2026

Primary problem statement	**Track 2a — PS 3: Multi-turn audits**
Secondary contributions	Track 2b — PS 9 and PS 10, subject to evidence
Model	CohereLabs/tiny-aya-global
Researcher / compute	Solo • 2 × Kaggle T4 GPUs
Dataset	20 pilot + 48 discovery + 24 held-out matched pairs
Submission deadline	14 November 2026 • end of day AoE

Executive decision

AyaTrace will investigate how instructions from untrusted documents, tool outputs and stored memory affect Tiny Aya Global over multi-turn conversations. It will attribute the first unsafe decision to its source and turn, identify candidate internal representations/components, and attempt a reversible targeted intervention. The project is framed primarily as a multi-turn audit (PS 2a-3); PS 2b-9/10 are evidence-dependent secondary contributions.

The project will start with one checkpoint, Tiny Aya Global. Base, Fire, Hindi/code-mixed extensions and a 4096-token boundary study are deferred until the main pipeline works.

1. Research question and hypotheses

Research question: At what point in a multi-turn conversation does untrusted content acquire causal influence over Tiny Aya's response, can that influence be traced to its original source, and can a targeted intervention reduce unsafe behavior without damaging legitimate instruction following?

- **H1 — Role confusion:** internal representation is affected by serialized role and writing style; source provenance and message role must be measured separately.
- **H2 — Late causal influence:** an injection may be decodable early but gain causal leverage at later layers.
- **H3 — Source-specific pathways:** document, tool and memory inputs may differ in serialization, position and internal influence.
- **H4 — Targeted intervention:** a candidate circuit/activation intervention may reduce unsafe outcomes while retaining benign utility.

These are hypotheses, not expected results. Null, diffuse or source-specific results are valid outcomes.

2. Scope and dataset allocation

The initial project uses Tiny Aya Global only. It is a 3.35B instruction-tuned checkpoint with an 8K input context and CC-BY-NC terms plus Cohere Labs' acceptable-use requirements. Record the immutable model revision and installed software versions.

Split	Retrieved document	Tool output	Memory	Total matched pairs
Pilot	7	7	6	20

Split	Retrieved document	Tool output	Memory	Total matched pairs
Discovery	16	16	16	48
Held out	8	8	8	24
Total	**31**	**31**	**30**	**92**

Each matched pair contains one clean and one corrupted condition: 184 episode records. The pilot is used for implementation/debugging and is not pooled into the final held-out evaluation. Discovery is used to select candidate layers/circuits/intervention settings. Held-out scenario families stay untouched until the analysis is frozen.

pair_id joins the clean and corrupted records. scenario_family_id groups related paraphrases/templates and is the unit used to prevent leakage across splits.

3. Threat model and annotation

An in-scope injection is an attacker-controlled or lower-trust directive that attempts to change the authorized task, override policy, disclose protected synthetic information, or trigger an unauthorized action. Maliciousness is not determined by toxicity or keywords alone. Quoted security examples are not attacks unless the scenario presents them as a control instruction to the agent.

Pre-run labels: attack_present, source, trust, instruction intent, conflict type and protected security property. Post-run outcomes: attack_success, canary leakage, simulated unauthorized action, human-adjudicated task-integrity violation and first unsafe turn. Over-refusal and benign completion are separately adjudicated. Never infer pre-run attack labels from model output.

Critical distinction: source provenance is independent of model-visible serialization. Record attack_source (document/tool/memory) separately from serialization_role (system/user/assistant/tool) and serialization_format (native tool message or application envelope). For memory, record both origin_turn/write time and retrieval_turn/exposure time.

All canaries are synthetic. Tools are sandboxed; no real external action is executed.

4. Source-specific episode runtime

- **Document:** application retrieves a document; its contents are inserted with an explicit source envelope. Preserve source ID and token span.
- **Tool output:** simulate a tool call/result. Probe Tiny Aya's exact chat template for native tool role support. For the initial benchmark, serialize tool results in a clearly labelled application-generated user envelope and report that it tests tool-output provenance, not native tool-role semantics. A native tool-role extension is optional and requires a verified complete assistant tool-call/result protocol; a bare role-template acceptance test is insufficient.
- **Memory:** store a record application-side at an earlier turn, then retrieve it later. The write event is not itself a model message. The retrieved content is serialized as an untrusted record; its provenance remains memory.

The previous scaffold's main methodological defect was converting all three sources into user messages without recording the distinction. The corrected design deliberately uses explicit application envelopes for all three source types in the initial benchmark, while retaining attack_source separately from serialization_role and serialization_format. This tests source provenance without claiming native tool-role semantics. A native tool-role extension is allowed only after implementing and verifying the full assistant tool-call/result protocol from Tiny Aya examples.

Multi-turn requirement: every episode must contain at least two assistant decision points after the relevant source is exposed. The source event and follow-up user turn must both be represented in the transcript. For memory, record the earlier application-side write and the later retrieval event. The starter fixtures in notebook v1.1 implement this minimum; they are smoke fixtures, not the full benchmark.

5. Experimental stages and gates

Stage A — Registration, environment and model gate

1. Confirm registration and Discord status. The hackathon website lists registration closing 5 November 2026; if not already registered, check the portal immediately and contact organizers if registration is no longer accepted.
2. Confirm Kaggle Python, PyTorch, CUDA and both T4 devices.
3. Load Tiny Aya Global on one T4 first; two cards do not aggregate VRAM.
4. Inspect the chat template, model revision, context limit, supported roles and architecture.
5. Install/pin CircuitKIT and run the current Tiny Aya real-weight parity and EAP-IG smoke tests from that exact commit.

Gate A: model loads; template and architecture are recorded; CircuitKIT parity passes locally. If the gate fails, debug or use custom HF hooks for patching/ablation; do not claim CircuitKIT support just because the README lists the architecture.

Stage B — Dataset and source-aware runner

6. Implement the 3 source pathways and separate provenance/serialization metadata.
7. Build the 20 pilot pairs (7/7/6) and manually review every example. Each fixture must contain at least two assistant decision points after source exposure. Each fixture must contain at least two assistant decision points after source exposure. Each fixture must contain at least two assistant decision points after source exposure.
8. Validate schema, pair integrity, source labels, scenario-family separation and token alignment.
9. Implement deterministic full/partial canary checking and explicit simulated-action checking.
10. Use CuratorKIT as a separate curation/provenance stage. Do not let toxicity filtering or deduplication overwrite the canonical matched pairs.

Gate B: 20 pilot pairs across all sources pass review, serialization tests and integrity validation.

Stage C — Behavioral baseline

11. Run clean and corrupted conditions with deterministic generation settings.
12. Record full transcripts, responses per turn, canary results, structured simulated actions and first failure turn.
13. Create 48 discovery and 24 held-out pairs (16/16/16 and 8/8/8). Split by scenario family.
14. Report source-wise attack success, canary leakage, unauthorized action, human-adjudicated task integrity, benign task completion and over-refusal.

Gate C: baseline is measured for all three sources and is not dependent on an LLM judge as the only ground truth.

Stage D — Mechanistic investigation

15. Build role probes on identical/control-matched text serialized under roles the chat template actually supports. Hold out content; vary style and role separately.
16. Capture residual-stream/block outputs, attention outputs, MLP intermediates and source-token positions. Record exact module paths and tensor semantics.
17. Run layer-wise activation-patching sweeps on a balanced subset. Patch final prompt position and injected source span. Use fixed matched transcript prefixes for causal comparisons because free-running trajectories may diverge. Run reverse, benign-donor, random/magnitude and no-patch controls.
18. Identify candidate causal layers; compare their effect to role-probe measurements.

Gate D: identify a causal window or report that no stable window was found. Activation differences alone are not enough.

Stage E — Circuit discovery and intervention

19. Run node-level EAP-IG on 12 balanced discovery pairs (4 per source) as the first feasibility run. Expand to 24+ only if stable and affordable.

20. Use a differentiable safe-vs-unsafe continuation objective for discovery. Do not treat a multi-token canary as a single-token logit target. Verify token boundaries; sequence log-probability is a possible proxy.
21. Evaluate the circuit with CircuitKIT's six pillars: patching, ablation, stability, robustness, baselines and generalization. Report signed/raw effects where available.
22. Attempt one reversible targeted intervention. Avoid permanent pruning as the first test.
23. Compare the original and intervened model on the same held-out set. Report safety outcomes and benign utility together.

Gate E: intervention effect is measured on held-out data, or the negative/null result is documented. PS 2b-9/10 claims are made only if supported by causal intervention results.

Stage F — Integration and release

24. Chain CuratorKIT → CircuitKIT → AuditKIT as a working pipeline. Toolkit APIs must be read from the pinned installed versions; do not use speculative constructors.
25. Publish synthetic dataset, schema, redacted traces/logs, aggregate metrics and circuit/intervention artifacts on Hugging Face as permitted.
26. Publish reproducible code/notebooks and a clear README on GitHub; add an HF Article and a Space if feasible/relevant.
27. Write the report and submission abstract; include multiple before/after metrics and exact model/repository revisions.
28. Confirm with organizers how to represent the gated Tiny Aya Global checkpoint in the required public HF model collection. Do not re-upload Cohere weights; publish only artifacts you created and are allowed to redistribute.

6. Metrics

Primary behavioral metrics: attack success rate; full/partial canary leakage; unauthorized simulated action rate; first failure turn; benign task completion; over-refusal. Provide denominators, uncertainty intervals and source-wise breakdowns.

Mechanistic metrics: role-probe accuracy/AUROC; layer-patching effect; activation attribution; circuit patching/ablation; stability/robustness/generalization. Clearly distinguish correlation, attribution and causation.

Discovery versus evaluation: circuit discovery uses a differentiable contrast (safe continuation log-probability minus unsafe continuation log-probability, or a verified single-token gap). Behavioral evaluation uses free generation plus deterministic canary and simulated-action checks. A discovery proxy is not a safety metric.

7. Compute plan for two Kaggle T4s

Use one T4 for the initial model load and short-context tests. Two T4s do not behave as one 32-GB device. Keep batch size 1 for activation work, start with roughly 1K–3K input tokens, cache selected layers/positions only, and record actual VRAM. The second T4 can host an independent worker or replicate only after verifying that two processes fit. CircuitKIT may load its own model representation; run baseline inference and CircuitKIT in separate sessions if simultaneous memory is a problem.

The fallback for expensive EAP-IG is custom layer patching and selected component interventions using verified Hugging Face hooks. This changes the method, not the model or research question.

8. Calendar

Date	Priority output
Nov 5–6	Registration status; access; model/template; CircuitKIT gate
Nov 7	Source-aware runner; 20 pilot pairs

Date	Priority output
Nov 8	Pilot evaluation; build 48 discovery + 24 held-out pairs
Nov 9	Curate/freeze dataset; full behavioral baseline
Nov 10	Role probes and activation capture
Nov 11	Layer-wise causal patching
Nov 12	EAP-IG and circuit faithfulness
Nov 13	Targeted intervention and held-out evaluation
Nov 14	Write-up, HF/GitHub artifacts, public demo/repo and submission

The hackathon site lists submissions due end of day 14 November 2026 AoE. Use the portal's displayed cutoff as authoritative and aim to submit in India time well before the last AoE hours. Registration is listed as closing 5 November 2026; if registration is not already complete, check the portal immediately and contact the organizers rather than assuming late registration is possible.

9. Risk register and fallback

Risk	Response
Registration not accepted	Check portal immediately; contact organizers; do not assume late registration is possible
CircuitKIT parity fails	Pin/inspect dependencies; use custom HF hooks for patching/ablation; disclose EAP-IG limitation
Tool role unsupported	Use explicit tool-result envelope; keep source provenance; narrow claims to application-level tool-output injection
T4 memory/runtime insufficient	Shorter contexts, batch 1, selected activation caches, discovery subset 12 pairs, sequential runs
EAP-IG too expensive	Manual layer sweep + targeted patch/ablation; do not invent circuit evidence
Diffuse or source-specific mechanisms	Report them; do not force a universal circuit
Intervention harms utility	Report signed effects and over-refusal; do not claim successful repair
Dataset too small	Call it a controlled pilot/benchmark prototype; report denominators and uncertainty

10. Claim boundaries

- Role-probe score is representational decodability, not proof of authority understanding.
- Layer activation difference is correlational.
- EAP-IG is attribution, not causal proof.
- Patching/ablation is causal evidence only relative to the tested metric, prompt distribution and controls.
- A discovered circuit is task/model/metric-specific.
- If native tool role is not supported, report the exact application serialization; do not claim native tool-call robustness.
- A successful intervention on synthetic data is not proof of production security. Runtime permissions and tool access controls remain necessary.

11. Submission checklist

- [ ] Registration/Discord status confirmed.
- [ ] Tiny Aya Global revision and license recorded.
- [ ] Two-GPU setup and per-GPU memory recorded.
- [ ] CuratorKIT, CircuitKIT and AuditKIT commits pinned.
- [ ] Source-aware runner and schema v1.0 validated.
- [ ] Pilot/discovery/held-out allocations complete.
- [ ] Baseline and multiple before/after metrics reported.
- [ ] Causal tests include controls and benign utility checks.
- [ ] Public GitHub repository with reproducibility instructions.
- [ ] Hugging Face dataset collection and logs/circuit artifacts public as permitted.
- [ ] HF Article or GitHub write-up and public demo/Space if relevant.
- [ ] Abstract under 500 words and final problem statements/toolkits listed.

References and what they establish

Source	What we use it for
Hackathon rules and problem statements — https://hackathon.lexsi.ai/	Track 2a PS 3 wording, Track 2b PS 9/10, model restrictions, repository chaining, Hugging Face and submission requirements.
Tiny Aya Global model card — https://huggingface.co/CohereLabs/tiny-aya-global	Checkpoint lineage, 3.35B size, 8K input context, chat-template example, CC-BY-NC terms and architecture notes.
Tiny Aya technical report — https://arxiv.org/abs/2603.11510	Model-family and training background.
CircuitKIT repository — https://github.com/Lexsi-Labs/CircuitKIT	Current Tiny Aya (cohere2) compatibility status, install constraints and supported discovery/evaluation/intervention surfaces; validate locally against a pinned commit.
CircuitKIT custom data — https://circuitkit.lexsi.ai/user-guide/custom-data/	Explicit clean/corrupt dataset setup and task-specific discovery data.
CircuitKIT evaluation — https://circuitkit.lexsi.ai/user-guide/evaluation/	Six-pillar faithfulness checks and evaluation configuration.
CuratorKIT — https://github.com/Lexsi-Labs/CuratorKIT	Dataset ingestion, curation, deduplication and provenance stage.
AuditKIT — https://github.com/Lexsi-Labs/AuditKIT	Whole-episode evaluation and diagnosis stage.
Localization paper — https://arxiv.org/abs/2609.37737	Prior evidence for early decodability and later causal leverage; motivates but does not predetermine Tiny Aya's result.
Prompt Injection as Role Confusion — https://arxiv.org/abs/2603.12277	Role probes and style/role confusion hypothesis; probes measure representational decodability, not authority understanding.
CircuitKIT paper — https://arxiv.org/abs/2607.19317	Circuit discovery, intervention and multi-pillar faithfulness evaluation background.